

ソフトウェア演習 2B 第 1 回レポート

学籍番号: 00000000 氏名: 豊橋 太郎

2026 年 6 月 23 日

1 課題 1

1.1 課題内容

リスト 1, 2 で実装した Message クラスに対してリスト 3 に示すプログラムを実行したところ、実行結果はリスト 5 のようにエラーが発生した。その理由を説明しなさい。

Listing 1 Message.cpp

```
1 #include <stdlib.h>
2 #include <string.h>
3 #include <string>
4 #include "Message.h"
5
6 Message::Message(): message(nullptr) {
7 }
8
9 Message::Message(const char* _message) {
10     message = new char [strlen(_message) + 1];
11     strcpy (message, _message);
12 }
13
14 Message::~Message() {
15     if (message != nullptr) delete [] message;
16 }
17
18 void Message::setMessage (const char* _message) {
19     if (message != nullptr) delete [] message;
20     message = new char [strlen(_message) + 1];
21     strcpy (message, _message);
```

```

22 }
23
24 char* Message::getMessage (void) {
25     return message;
26 }
27
28 std::istream& operator>>(std::istream& stream, Message& obj) {
29     std::string buffer;
30     std::getline(stream, buffer);
31     obj.setMessage(buffer.c_str());
32     return stream;
33 }
34
35 std::ostream& operator<<(std::ostream& stream, Message& obj) {
36     stream << obj.getMessage();
37     return stream;
38 }

```

Listing 2 Message.h

```

1 #include <iostream>
2
3 class Message {
4 private:
5     char* message;
6
7 public:
8     Message();
9     Message(const char* _message);
10    ~Message();
11
12    void setMessage (const char* _message);
13    char* getMessage (void);
14 };
15
16 std::istream &operator>>(std::istream& stream, Message& obj);
17 std::ostream &operator<<(std::ostream& stream, Message& obj);

```

Listing 3 main.cpp

```

1 #include <iostream>
2 #include "Message.h"
3
4 int main (int argc, char *argv[]) {
5     Message obj1("Hello World.");

```

```

6   Message obj2 = obj1;
7   std::cout << obj2 << std::endl;
8
9   return 0;
10  }

```

また、以下に printf 関数を用いて、コピーしたオブジェクトのメンバ変数 message のそれぞれのアドレスを調べた。その際の main.cpp の変更点をリスト 4 に示す。実行結果をリスト 5 に示す。

Listing 4 main.cpp (アドレス確認用)

```

1   ...
2
3   std::cout << obj2 << std::endl;
4   printf("obj1: %p\n", (void*)obj1.getMessage()); //追加
5   printf("obj2: %p\n", (void*)obj2.getMessage()); //追加
6
7   return 0;
8   }

```

Listing 5 実行結果

```

1   $ ./main
2   Hello World.
3   obj1: 0xa000005c0
4   obj2: 0xa000005c0
5   Aborted

```

1.2 プログラムに関する説明

リスト 5 の実行結果より、aborted によってエラーで終了していることがわかる。また、printf の出力より obj1 と obj2 の message メンバ変数が同じアドレスを指していることがわかる。よって、プログラム終了時の aborted によるエラーは二重解放の発生が原因であることがわかる。

リスト 1 とリスト 2 で実装した Message クラスにおいて、コピーコンストラクタの実装をしていないため、デフォルトのコピーコンストラクタが呼び出されている。そのため、メンバ変数 message はポインタ型のため、obj2 の message は obj1 の message メモリをコピーしてしまう。そのため、メモリを解放しようとした際にメモリを二重で開放してしまうと考えられる。

リスト 4 の getMessage() の呼び出し時に (void*) を明示したのは、printf 関数のフォーマット指定子に%p を使用する際に、引数として渡すポインタ型の値は (void*) であるからである。(void*) をつけなくても動作はするが、正確には方の不整合が発生してしまっているため、明示的に (void*) をつけている。

2 課題 2

2.1 課題内容

Message クラスに対するコピーコンストラクタを実装して、リスト 3 のプログラムを実行してもエラーがでないようにしなさい。

2.2 ソースコード

以下のリスト 6 とリスト 7 にコピーコンストラクタを実装した Message クラスのソースコードを示す。ただし、ほとんど課題 1 と同じソースを使用しているため、変更部分をピックアップして示す。

Listing 6 Message.h (課題 2)

```
1 ...
2
3 public:
4     Message();
5     Message(const char* _message);
6     Message(const Message& obj); // コピーコンストラクタを追加
7     ~Message();
8
9     ...
```

Listing 7 Message.cpp (課題 2)

```
1 ...
2
3 Message::Message(const char* _message) {
4     message = new char [strlen(_message) + 1];
5     strcpy (message, _message);
6 }
7
8 // コピーコンストラクタを追加
9 Message::Message(const Message& obj){
10     if(obj.message != nullptr){
```

```

11     message = new char [strlen(obj.message) + 1];
12     strcpy (message, obj.message);
13 } else {
14     message = nullptr;
15 }
16 }
17
18 Message::~Message() {
19     if (message != nullptr) delete [] message;
20 }
21 ...

```

プログラムの実行においては、リスト 3 のプログラムをそのまま使用した。以下にその実行結果をリスト 8 に示す。

Listing 8 実行結果 (課題 2)

```

1 $ ./main
2 Hello World.
3 obj1: 0xa000005c0
4 obj2: 0xa00000490

```

2.3 プログラムに関する説明

リスト 8 の実行結果より、Aborted によるエラーが発生していないことがわかる。また、printf の出力より obj1 と obj2 の message メンバ変数が異なるアドレスを指していることがわかる。よって、二重解放のエラーが発生していないことがわかる。

リスト 6 とリスト 7 での変更点より、本課題では Message クラスにコピーコンストラクタを実装した。コピーコンストラクタ内では、コピー元のオブジェクトのメンバ変数 message の文字列をコピーして、コピー先で新しく確保したメモリに格納している。そうすることで、コピー元とコピー先のオブジェクトのメンバ変数 message が異なるアドレスを指すようになり、二重解放のエラーが発生しないようになった。

Listing 9 Message.cpp (課題 2 のコピーコンストラクタのエラー例)

```

1 ...
2 // if分を削除
3 Message::Message(const Message& obj){
4     message = new char [strlen(obj.message) + 1];
5     strcpy (message, obj.message);
6 }
7 ...

```

Listing 10 main.cpp (課題 2 のコピーコンストラクタのエラー例)

```
1 #include <iostream>
2 #include "Message.h"
3
4 int main (int argc, char *argv[]) {
5     Message obj1(nullptr);
6     Message obj2(obj1);
7     std::cout << obj2 << std::endl;
8     printf("obj1: %p\n", (void*)obj1.getMessage());
9     printf("obj2: %p\n", (void*)obj2.getMessage());
10
11     return 0;
12 }
```

Listing 11 実行結果 (segmentation fault)

```
1 $ ./main
2 Segmentation fault
```

コピーコンストラクタ内で、if 分によってコピー元のメンバ Message のオブジェクト message が nullptr である場合の処理を実装している。リスト 9 のように if 分の処理を追加しないと、リスト ?? のように main 関数でデフォルトコンストラクタを呼び出してしまうと、コピー先のオブジェクトのメンバ変数 message が不定値を指すことになってしまうため、実行結果 11 のように segmentation fault が発生するため、この処理を追加している。

3 課題 3

3.1 課題内容

(課題内容を記述)

3.2 ソースコード

Listing 12 main.cpp (課題 3)

```
1 // 課題3のソースコードを記述
```

3.3 プログラムに関する説明

(作成したプログラムに関する説明を記述)

4 課題 4 チェック項目への回答

1. private, protected, public の違いについて
(ここへ回答を記述します)
2. その他のチェック項目
(ここへ回答を記述します)